



Data Structures And Algorithms

Name : Samreen Zia
 Submitted to : Mr. Rehan Ahmed
 Semester : BSE (3A)
 Date : 20- Oct- 2025

LAB NO 4

Task 1 :

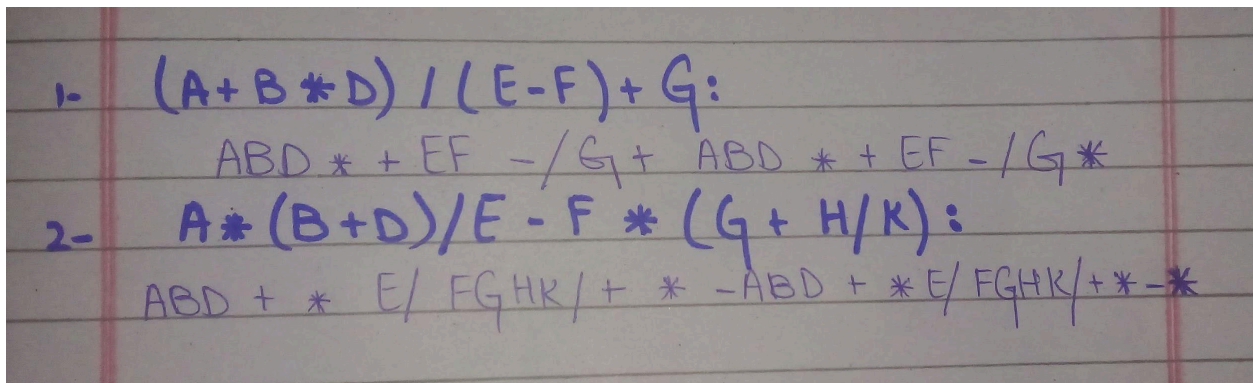
Give answers to the following.

1. Convert (manually) the following expressions to postfix.

$(A+B*D)/(E-F)+G$: _____

$A*(B+D)/E-F*(G+H/K)$: _____

Solution:



2. Evaluate the given Postfix expression and trace the contents of the Stack at each step using the standard evaluation algorithm. $2\ 7\ 3\ -\ /\ 2\ 1\ 5\ +\ * +$

Symbol	Stack Contents
--------	----------------

2

7

3

-

/
 2
 1
 5
 +
 *
 +
 Result : _____

Solution:

Symbols	Stack Contents
2	[2]
7	[2, 7]
3	[2, 7, 3]
-	[2, 4]
/	
2	
1	
5	
+	
*	
+	
Final	Result : 12.5

3. Convert the following expression from infix to postfix and show the contents of Stack and the output expression at each step. $(A+B) * C - D+F*G$

Symbol.	Stack	Contents	Output Expression
---------	-------	----------	-------------------

(
 A
 +
 B
)
 *
 C
 -
 D
 +
 F

*
G

Solution:

Symbol	Stack Contents	Output Expression
(	[(]	
A	[(]	A
+	[(] [+]	A
B	[(] [+]	AB
)	[]	AB+
*	[*]	AB+
C	[*]	AB+C
-	[-]	AB+C *
D	[-]	AB+C * D
+	[+]	AB+C * D -
F	[+]	AB+C * D - F
*	[+ , *]	AB+C * D - F
G	[+ , *]	AB+C * D - FG
	[]	AB+C * D - FG * +

Conclusion :
The postfix expression is :
 $AB + C * D - FG * + AB + C * D - FG * + *$

Code Task # 01

Implement the algo to evaluate the postfix expression using a Stack and display the result. (For simplicity, assume single digit numbers in the expression.)

23+5*6+

Note: Use existing stack class #include<stack>

Program

```
#include <iostream>
#include <stack>
#include <string>
```

```
// Function to evaluate postfix expression
int evaluatePostfix(const std::string& expression) {
    std::stack<int> stack;

    for (char c : expression) {
        // If the character is a digit, push it to the stack
    }
```

```

    if (c >= '0' && c <= '9') {
        stack.push(c - '0'); // Convert char to int
    }
    // If the character is an operator, pop two operands and apply the operation
    else if (c == '+' || c == '-' || c == '*' || c == '/') {
        int operand2 = stack.top();
        stack.pop();
        int operand1 = stack.top();
        stack.pop();

        switch (c) {
            case '+':
                stack.push(operand1 + operand2);
                break;
            case '-':
                stack.push(operand1 - operand2);
                break;
            case '*':
                stack.push(operand1 * operand2);
                break;
            case '/':
                stack.push(operand1 / operand2);
                break;
        }
    }
}
// The final result is the only element left in the stack
return stack.top();
}

int main() {
    std::string expression = "23+5*6+";
    int result = evaluatePostfix(expression);
    std::cout << "Result: " << result << std::endl;
    return 0;
}

```

Output

```
Result: 31
```

Code Task # 02

Implement the algo to convert the infix expression to postfix and display the result on screen

Note: Use existing stack class #include<stack>

Program

```
#include <iostream>
#include <stack>
#include <string>

// Function to check if a character is an operator
bool isOperator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '^');
}

// Function to get the precedence of an operator
int getPrecedence(char c) {
    switch (c) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        case '^':
            return 3;
        default:
            return 0;
    }
}

// Function to convert infix expression to postfix
std::string infixToPostfix(const std::string& expression) {
    std::stack<char> stack;
    std::string postfix;

    for (char c : expression) {
        // If the character is an operand, add it to the postfix expression
        if (c >= 'a' && c <= 'z' || c >= 'A' && c <= 'Z' || c >= '0' && c <= '9') {
            postfix += c;
        }
        // If the character is '(', push it to the stack
        else if (c == '(') {
            stack.push(c);
        }
    }
}
```

```

        // If the character is ')', pop operators from the stack and add them to the postfix expression
until '(' is found
    else if (c == ')') {
        while (stack.top() != '(') {
            postfix += stack.top();
            stack.pop();
        }
        stack.pop(); // Remove '(' from the stack
    }
    // If the character is an operator
    else if (isOperator(c)) {
        // Pop operators from the stack and add them to the postfix expression until the stack is
empty or the top of the stack has lower precedence
        while (!stack.empty() && getPrecedence(c) <= getPrecedence(stack.top())) {
            postfix += stack.top();
            stack.pop();
        }
        stack.push(c); // Push the current operator to the stack
    }
}

// Pop any remaining operators from the stack and add them to the postfix expression
while (!stack.empty()) {
    postfix += stack.top();
    stack.pop();
}
return postfix;
}

int main() {
    std::string expression = "a+b*c-(d/e+f)";
    std::string postfix = infixToPostfix(expression);
    std::cout << "Infix Expression: " << expression << std::endl;
    std::cout << "Postfix Expression: " << postfix << std::endl;
    return 0;
}

```

Output

```

Infix Expression: a+b*c-(d/e+f)
Postfix Expression: abc*+de/f+-

```